Sistema de Control Académico

Documentación del avance de la segunda fase

Asignatura: Desarrollo de Aplicaciones con Software Propietario [DSP404].
Institución: Universidad Don Bosco, Escuela de Computación.
Fecha de preparación: 21 de octubre de 2026.

El proyecto continúa el registro de estudiantes, docentes, cursos, matrículas y calificaciones modelado en fase 1. La ampliación incorpora una aplicación ASP.NET Core MVC con persistencia SQLite, CRUD, arquitectura en capas y una API de consulta académica.

Autor del informe

José Miguel Sosa Ayala
Carné: SA251597

Objeto del documento

Este informe documenta la ampliación implementada en la segunda fase del Sistema de Control Académico. Describe los componentes reutilizados del primer avance, su transformación en una aplicación web, la persistencia de datos y las funciones incorporadas en el repositorio de GitHub.

El primer avance aportó las entidades, las relaciones principales, el prototipo de consola y la propuesta visual. En la segunda fase se incorporaron páginas ASP.NET funcionales, operaciones CRUD, acceso a base de datos y un servicio de consulta académica.

Repositorio: https://github.com/HISTOR126/ControlAcademico-Fase2.
Rama de implementación: fase2-web-aspnet
Propuesta de cambios: https://github.com/HISTOR126/ControlAcademico-Fase2./pull/1

Ampliación implementada

La aplicación transforma el prototipo de consola en páginas web funcionales de las mismas entidades. Los datos nuevos se guardan en un archivo SQLite y permanecen después de reiniciar el servidor.

Requisito	Implementación
Persistencia y CRUD	EF Core 10.0.12 y SQLite; agregar, consultar, modificar y eliminar las cinco entidades.
Interfaz ASP.NET	Panel principal, listados con búsqueda, formularios, detalles y confirmación de eliminación en Razor.
Validación y errores	Validaciones de formularios y servidor, reglas de negocio, restricciones de BD, rollback y mensajes controlados.
Arquitectura	Presentación, negocio y datos; contrato y entidades compartidas en dominio.
Servicio web	GET /api/estudiantes/{carnet}/situacion, con matrículas, notas y resultado en JSON.
Documentación	README con instrucciones y capturas, modelo relacional, esquema SQL y resultados de pruebas.

Reglas principales

Una matrícula activa no puede repetirse para el mismo estudiante y curso. El curso debe tener cupo. Una matrícula con nota no puede cancelarse ni cambiar de estudiante o curso. Las notas admiten valores entre 0 y 10 y se aprueban desde 6.

La eliminación de un registro con dependencias se rechaza. La eliminación se realiza respetando las dependencias: primero la nota, después la matrícula y luego los registros relacionados.

Límites funcionales: una nota final por matrícula; CUM registrado manualmente; sin períodos académicos, usuarios autenticados ni cálculo automático de ponderaciones. Estas funciones adicionales no se especifican en el documento recibido.

Arquitectura en capas

Capa / proyecto	Responsabilidad
Presentación / Web	Controladores MVC, vistas Razor, formularios y endpoint API. Recibe entradas y presenta resultados.
Lógica / Negocio	AcademicoService coordina operaciones y valida existencia, duplicados, cupos, notas, dependencias y versiones.
Acceso / Datos	RepositorioAcademico y AcademicoDbContext ejecutan consultas y transacciones con EF Core.
Dominio compartido	Entidades de fase 1 y contrato IRepositorioAcademico. Mantiene las reglas base y separa el almacenamiento.

Recorrido de una matrícula

1. El usuario completa el formulario ASP.NET y envía la solicitud.
2. El controlador valida el modelo de entrada y llama al negocio.
3. Negocio comprueba estudiante, curso, matrícula y cupo.
4. El repositorio aplica el cambio en una transacción SQLite.
5. El controlador redirige al detalle o muestra el error en el formulario.

Las páginas y los controladores no consultan directamente la base de datos. La configuración de dependencias se realiza en el arranque de la aplicación.

Reutilización de la fase 1

Se conservan las clases Persona, Estudiante, Docente, Curso, Matrícula y Calificación, junto con las interfaces y la consola. La web reutiliza constructores del dominio para validar personas y cursos; utiliza registros de persistencia para almacenar las entidades sin acoplar el prototipo a EF Core.

Control de consistencia

Cada escritura utiliza una transacción serializable. Los códigos únicos y claves foráneas respaldan las validaciones. La propiedad Version detecta ediciones de formularios obsoletos; los conflictos se muestran como mensajes y la operación se revierte.

Modelo de datos

Tabla	Datos principales	Relaciones
Estudiantes	Carnet, nombres, correo, teléfono, nacimiento, carrera, facultad, ciclo y CUM.	Un estudiante puede tener varias matrículas.
Docentes	Código, nombres, contacto, nacimiento y especialidad.	Un docente puede tener varios cursos.
Cursos	Código, nombre, créditos, cupo máximo y docente.	Cada curso pertenece a un docente.
Matrículas	Código, estudiante, curso, fecha UTC y estado.	Relaciona estudiantes y cursos.
Calificaciones	Matrícula y nota final.	Como máximo una nota por matrícula.

Todas las tablas tienen ID y Version. Se incluyen índices únicos para carnet, códigos, matrícula activa estudiante-curso y nota por matrícula. Las relaciones usan borrado restringido para impedir registros huérfanos.

Creación y persistencia

Al iniciar, EF Core crea el esquema inicial mediante EnsureCreated. El archivo predeterminado es App_Data/academico.db dentro del proyecto web. El esquema SQL está en database/esquema.sql. No se necesita un servicio de SQL Server para esta demostración.

Los datos de ejemplo se crean solamente cuando la base está vacía: DOC-001, CT-0001, CT-0002, DSP404, MAT-0001 y nota 8.5. Los ejemplos no se vuelven a cargar sobre datos modificados.

Versiones futuras: esta entrega no incluye migraciones incrementales. Si se cambia el modelo después de usar la aplicación, deberá diseñarse una migración con respaldo para conservar los datos.

Interfaz y servicio web

Las páginas funcionales conservan la propuesta visual morada de la fase 1. Cada módulo incluye listado, alta, edición, detalle y confirmación de eliminación. El panel obtiene indicadores reales desde la base de datos.

Pantalla	Función
Principal	Indicadores y acceso a matrículas recientes.
Listado	Buscar por nombre o código y acceder a detalle, edición y eliminación.
Formulario	Capturar datos con campos obligatorios, límites numéricos y mensajes de servidor.
Detalle	Consultar datos y navegar por relaciones estudiante-curso-matrícula.
Eliminar	Confirmar la operación y comprobar dependencias antes de borrar.

API académica

Consulta: GET /api/estudiantes/CT-0001/situacion

La respuesta contiene carnet, nombre, apellido y una lista de matrículas con código, curso, estado activo, nota y resultado. No devuelve correo, teléfono ni fecha de nacimiento.

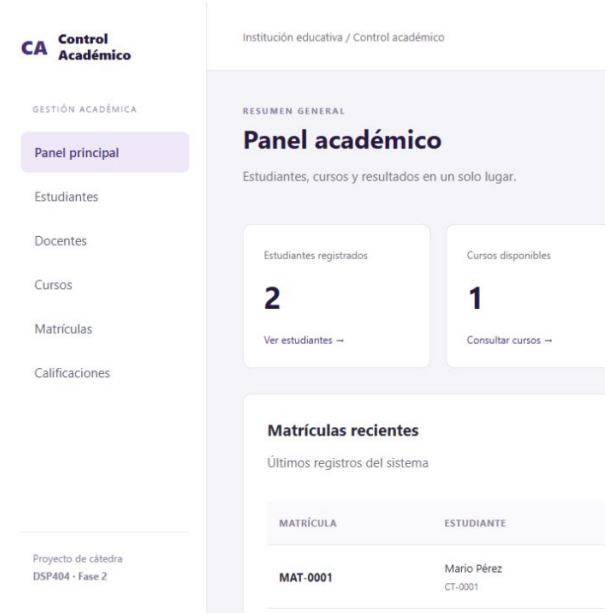
```
{ "carnet": "CT-0001",
  "nombre": "Mario",
  "matriculas": [
   { "codigo": "MAT-0001", "curso": "DSP404",
    "activa": true, "nota": 8.5,
    "estado": "Aprobado" }
  ] }
```

Respuesta 200 para un estudiante existente, 404 si no existe y 503 si se produce un error controlado de consulta. El endpoint es de solo lectura y utiliza la misma capa de negocio.

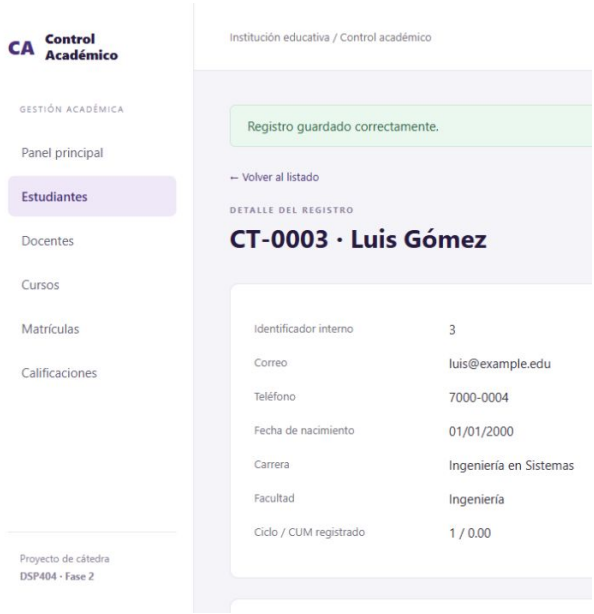
Verificación y capturas

Compilación: solución Release, 0 errores y 0 advertencias.
Dominio: 14 casos aprobados.
SQLite: 22 comprobaciones aprobadas.
HTTP: 25 páginas, CRUD, API, validación, antiforgery y persistencia después de reiniciar el proceso web.
Todas las comprobaciones ejecutadas pasaron.

Se comprobó también un alta de estudiante desde el navegador. Las capturas corresponden a la ejecución local y muestran el área visible de la ventana de prueba.



Panel principal



Detalle tras guardar un estudiante

Los scripts de pruebas se entregan con el código. Las pruebas HTTP utilizan una base temporal y no eliminan los datos de una base de usuario. No se ha verificado alojamiento público ni carga masiva multiusuario.

Ejecución y resultado del avance

La segunda fase incorpora una solución web ejecutable con el SDK .NET 10. Desde la raíz del repositorio se restauran las dependencias, se compila y se inicia la aplicación:

```
dotnet restore ControlAcademico.slnx
dotnet build ControlAcademico.slnx -c Release
dotnet run --project src/ControlAcademico.Web
  -c Release --no-build -- --urls http://localhost:5188
```

El comando de inicio se escribe en una sola línea. La aplicación se consulta en http://localhost:5188. La primera restauración descarga los paquetes requeridos desde NuGet; posteriormente, los registros se guardan en el archivo SQLite local.

Resultado de la remodelación

El sistema pasó de un prototipo de consola con listas en memoria a una aplicación web con información persistente. Los módulos de estudiantes, docentes, cursos, matrículas y calificaciones incluyen registro, consulta, edición y eliminación, con pantallas de detalle y comprobación de dependencias.

La lógica de la aplicación se separó de las vistas y del acceso a datos. Las reglas de matrícula y calificación se aplican en la capa de negocio, mientras que las consultas y transacciones se realizan en la capa de datos. La API permite consultar notas y matrículas por carnet.

Documentación incorporada en GitHub

El repositorio incorpora instrucciones de ejecución, capturas reales de las pantallas, descripción de arquitectura, modelo relacional, esquema SQL y evidencias de pruebas. La implementación se organizó en dos commits: uno para el código de la aplicación y otro para documentación, capturas e informe.

La compilación y las pruebas automatizadas de GitHub Actions finalizaron correctamente para la versión publicada en la rama fase2-web-aspnet. La propuesta de cambios número 1 reúne la ampliación para su revisión e incorporación.

Base técnica utilizada: entidades y prototipo del primer avance de Control Académico. **Alcance documentado:** trabajo incorporado en la segunda fase; la consola anterior permanece como antecedente ejecutable.